

LEGALEASE PROJECT DOCUMENTATION

AI-POWERED LEGAL DOCUMENT GENERATOR

TEAM ID: SWTID-2026-8781
TEAM SIZE: 5

TEAM LEADER: KOUSHIK S

TEAM MEMBERS
DHANUSH S
SARGUNANITHI S
HARRY RITHIK P
PRITAM DEBNATH

GENAI • FASTAPI • STREAMLIT • GEMINI • DOCUMENT AUTOMATION

ABSTRACT

LegalEase is designed to simplify legal-document drafting through generative artificial intelligence. The system accepts structured information such as document type, parties, terms, and effective dates, then generates a formatted draft that can be previewed, edited, and exported. The documented architecture uses Streamlit for the frontend, FastAPI for the backend, Gemini for AI generation, and python-docx/FPDF utilities for document output.

The project combines automation with user control. Users can review generated content, modify it, and save the result as TXT, DOCX, or PDF. Branding features such as logos, headings, tables, and footers are included in the document-generation workflow.

DOCUMENT CONTENTS

- 1. INTRODUCTION
- 2. PROJECT OVERVIEW
- 3. PROBLEM STATEMENT
- 4. OBJECTIVES
- 5. SYSTEM REQUIREMENTS
- 6. TECHNOLOGY STACK
- 7. SYSTEM ARCHITECTURE
- 8. AI MODEL SELECTION AND PROMPTING
- 9. BACKEND API DESIGN
- 10. FRONTEND DESIGN AND USER EXPERIENCE
- 11. DOCUMENT FORMATTING AND EXPORT
- 12. CORE FUNCTIONS AND UTILITIES
- 13. WORKFLOW AND USER OPERATION
- 14. TESTING AND VALIDATION
- 15. DEPLOYMENT
- 16. SECURITY, PRIVACY AND RESPONSIBLE USE
- 17. LIMITATIONS AND FUTURE ENHANCEMENTS
- 18. CONCLUSION

1. INTRODUCTION

LegalEase is an AI-powered legal document generation platform designed to simplify the creation of structured legal documents. The project combines a Streamlit user interface, a FastAPI backend, and a Gemini-based generative AI component to transform user-provided information into editable legal content.

The system is intended for entrepreneurs, professionals, freelancers, landlords, employees, and individuals who need structured legal documents without requiring a legal background. Supported document scenarios include employment contracts, lease agreements, non-disclosure agreements, contracts, and other agreements.

LegalEase focuses on accessibility, structured generation, professional formatting, editing, and multi-format export. The source project documentation specifies TXT, DOCX, and PDF outputs, together with branding elements such as logos, headings, tables, and footers.

AREA	DOCUMENTED FOCUS
PURPOSE	ACCESSIBLE, STRUCTURED, EDITABLE LEGAL-DOCUMENT GENERATION

2. PROJECT OVERVIEW

The central purpose of LegalEase is to provide a guided workflow for entering legal-document requirements and receiving a generated draft. Users provide the document type, parties involved, terms and conditions, and effective date. These inputs are passed through the backend to the AI generation layer.

The generated content is returned to the interface for preview and editing. The application then provides document export options. This workflow creates a practical bridge between user-friendly data entry and professional document formatting.

The project documentation identifies three representative scenarios: a startup founder generating an employment contract, a freelancer generating an NDA, and a landlord generating a residential lease agreement. These scenarios demonstrate how the same workflow can support different document types and user needs.

3. PROBLEM STATEMENT

Creating legal documents manually can require familiarity with formal language, document structure, clauses, and formatting conventions. Users may also need to repeatedly prepare similar documents with different parties, dates, and terms.

LegalEase addresses this workflow challenge by collecting structured information through a simple interface and using generative AI to create a formatted draft. The application also reduces repetitive formatting work by supporting automated DOCX and PDF generation.

The project is positioned as a document-generation and accessibility solution. It is designed to help users understand and prepare structured legal content; generated material should still be reviewed appropriately before real-world legal use.

4. OBJECTIVES

The primary objective is to build an AI-assisted platform that generates customizable legal-document drafts from user inputs.

Specific objectives include: implementing a Streamlit frontend; developing a FastAPI backend; integrating Gemini for text generation; validating request data with Pydantic; providing editable previews; generating TXT, DOCX, and PDF outputs; supporting logos, headings, terms tables, headers and footers; and preparing the application for local deployment.

The project also emphasizes a user-centric workflow so that a person can enter document requirements, generate content, edit it, preview the result, and save it in a suitable format.

5. SYSTEM REQUIREMENTS

The project documentation lists Python 3.10+ as the recommended development environment and identifies FastAPI, Uvicorn, Streamlit, Google Generative AI SDK, python-docx, FPDF, Pillow, requests, and python-dotenv among the required packages.

A Gemini API key is configured through an environment file. The backend requires FastAPI and Uvicorn for API service execution, while Streamlit provides the interactive frontend. python-docx and FPDF support document export, and Pillow can support image-related processing such as branding assets.

A virtual environment is recommended to isolate project dependencies. A requirements.txt file can be used to install the application dependencies consistently.

6. TECHNOLOGY STACK

Frontend — Streamlit: The source documentation specifies Streamlit for user input, previews, editing, and download controls. Its layout facilities are used to create a responsive, user-friendly interface.

Backend — FastAPI: FastAPI provides the API layer and exposes the /generate endpoint. It receives structured JSON input and routes the request to the AI generation component.

AI Layer — Gemini: The source documentation selects gemini-1.5-pro for structured legal-document generation and describes its long context window, structured prompting capabilities, and suitability for formal text generation.

Document Processing — python-docx, FPDF and TXT: These components support Word, PDF, and plain-text output. The documentation also describes HTML-based preview rendering inside Streamlit.

Configuration — python-dotenv: Environment variables are used to keep the Gemini API key outside the application source code.

AREA	DOCUMENTED FOCUS
STACK	STREAMLIT + FASTAPI + GEMINI + PYTHON DOCUMENT UTILITIES

7. SYSTEM ARCHITECTURE

LegalEase follows a layered architecture consisting of the presentation layer, API/backend layer, AI generation layer, and document-formatting layer.

The Streamlit frontend collects document_type, parties, terms, and dates. A request is sent to the FastAPI backend. The backend validates the request using a Pydantic DocumentRequest model and invokes GeminiDocumentGenerator. The generated text is returned to the frontend, where it can be previewed, edited, and exported.

The architecture separates user interaction from API processing and AI generation. This modular approach also makes it easier to replace or extend individual components in future versions.

AREA	DOCUMENTED FOCUS
ARCHITECTURE	FRONTEND → API → AI CORE → DOCUMENT FORMATTING

8. AI MODEL SELECTION AND PROMPTING

The project documentation identifies Gemini 1.5 Pro as the selected model for the AI generation layer. The selection is based on the model's ability to handle structured text, formal documents, long contextual inputs, and dynamically generated clauses.

The GeminiDocumentGenerator is responsible for forming a structured prompt and sending it to Gemini through `generate_content`. The prompt incorporates the user's document type, parties, terms, and effective date so that the generated output is tailored to the request.

The AI layer should be treated as a generation component rather than an independent legal authority. The application workflow therefore benefits from editable output and user review before documents are used externally.

9. BACKEND API DESIGN

The FastAPI application is initialized in `main.py`. The root endpoint provides a simple service check, while routes from `routes.py` are included in the application.

`routes.py` defines the `DocumentRequest` Pydantic model and the `POST /generate` endpoint. The endpoint accepts `document_type`, `parties`, `terms`, and `dates`, validates the incoming structure, invokes `GeminiDocumentGenerator`, and returns the generated legal content.

This separation between `main.py` and `routes.py` keeps application initialization distinct from request-handling logic. It also provides a foundation for adding future API endpoints without placing all functionality in one file.

AREA	DOCUMENTED FOCUS
API	POST /GENERATE WITH PYDANTIC VALIDATION

10. FRONTEND DESIGN AND USER EXPERIENCE

The Streamlit interface is designed around a straightforward generation workflow. The source documentation describes a centered layout, logo placement, title area, input fields, generation controls, preview, editing, and download options.

The principal inputs are document type, parties involved, terms and conditions, and effective date. Terms can be entered as semicolon-separated clauses so that they can be represented as bullet-style items or used to construct a terms table in exported documents.

After generation, the user can view the result in a styled HTML preview and use an edit control to modify the generated text. The design therefore combines automation with direct user control over the final draft.

11. DOCUMENT FORMATTING AND EXPORT

LegalEase supports three documented export formats: TXT, DOCX, and PDF. TXT provides a plain-text representation, DOCX provides editable Word formatting, and PDF provides a branded, ready-to-use presentation.

The DOCX formatter uses `python-docx` to create titles, paragraphs, branding, and a terms table. The project documentation specifies Times New Roman for the generated Word document and describes logo and footer support.

The PDF formatter uses `FPDF` and is described as providing a centered logo, bold section headings, bullet-style terms, and headers/footers. HTML preview formatting is used to present generated content inside the Streamlit interface.

AREA	DOCUMENTED FOCUS
OUTPUT	TXT + DOCX + PDF WITH FORMATTING AND BRANDING

12. CORE FUNCTIONS AND UTILITIES

`sanitize_text(text)` removes special characters and typographic quotes to help maintain clean output formatting.

`format_docx(text, doc_type)` converts generated text into a Word document, including titles, paragraph formatting, logo placement, and an automatically generated terms table based on semicolon-separated input.

`format_pdf(text, doc_type)` creates a PDF with branding, headings, bullet-style terms, and footer information.

`format_html_preview(text)` converts generated output into styled HTML blocks for inline display in the Streamlit application.

Together, these utilities form the document-processing layer between AI-generated text and user-facing downloadable files.

13. WORKFLOW AND USER OPERATION

Step 1 — Select the required document type, such as Agreement, Contract, NDA, Lease Agreement, or Employment Offer Letter.

Step 2 — Enter the parties involved and their roles. The input may contain individuals or organizations such as a service provider and client or tenant and landlord.

Step 3 — Enter the required terms and conditions. The project documentation recommends separating bullet-style clauses with semicolons.

Step 4 — Provide the effective date.

Step 5 — Select Generate Document. The frontend sends the request to FastAPI, which invokes the Gemini generation layer.

Step 6 — Preview and edit the generated content. Step 7 — Export the final draft as TXT, DOCX, or PDF.

14. TESTING AND VALIDATION

Testing is intended to verify end-to-end functionality across input collection, API communication, AI generation, preview, editing, and document download.

The documented test flow includes entering document type, parties, terms, and date; generating content through Gemini; previewing the result; editing the generated text; and checking TXT, DOCX, and PDF downloads.

Validation should include checking the presence and formatting of logos, footers, headings, bullet points, terms tables, and correctly embedded user-provided terms. Different combinations of document types and inputs should also be tested as part of Gemini integration verification.

Because generated legal language can vary, functional testing should focus on workflow correctness, output structure, formatting, and successful API communication rather than assuming that every generated clause is legally appropriate for every jurisdiction or circumstance.

AREA	DOCUMENTED FOCUS
VALIDATION	END-TO-END WORKFLOW AND FORMAT CHECKS

15. DEPLOYMENT

The source documentation describes local deployment using FastAPI for the backend and Streamlit for the frontend. A virtual environment is created, dependencies are installed from requirements.txt, and the Gemini API key is stored in an environment file.

The backend can be launched with Uvicorn, while the Streamlit application is started separately. The documentation also identifies possible future deployment options including Render, Railway, Deta, Streamlit Community Cloud, Fly.io, or a VPS, depending on the deployment architecture.

A production deployment should additionally protect API credentials, validate user input, apply appropriate access controls, and review data-handling requirements before exposing the service to real users.

AREA	DOCUMENTED FOCUS
DEPLOYMENT	VIRTUAL ENVIRONMENT + REQUIREMENTS + ENVIRONMENT VARIABLES

16. SECURITY, PRIVACY AND RESPONSIBLE USE

Legal documents can contain sensitive information such as names, addresses, contract terms, and business information. The project documentation emphasizes secure handling of sensitive user data, and environment variables are used for API-key configuration.

API credentials should never be hard-coded into source files or committed to public repositories. Input validation and controlled API access are important for a deployed service.

LegalEase should be presented as an AI-assisted drafting and understanding tool rather than a substitute for qualified legal advice. Users should review generated documents and obtain appropriate professional review when required.

17. LIMITATIONS AND FUTURE ENHANCEMENTS

The documented implementation centers on generation, preview, editing, formatting, and export. Future enhancements identified in the project material include deeper contract analysis, integration with legal databases, and personalized recommendations.

Additional engineering improvements could include richer document templates, stronger validation, authentication, persistent document history, multilingual workflows, more advanced clause analysis, and improved document comparison capabilities. These are proposed extensions rather than features established as complete in the supplied documentation.

The project can also evolve toward broader deployment by improving scalability, observability, error handling, and privacy controls while maintaining the modular architecture.

AREA	DOCUMENTED FOCUS
FUTURE	DEEPER ANALYSIS, LEGAL DATABASES, PERSONALIZATION

18. CONCLUSION

LegalEase demonstrates how generative AI can be combined with a practical web interface and document-processing libraries to simplify legal-document drafting. Its architecture connects Streamlit, FastAPI, Gemini, and document-formatting utilities into a single workflow.

The system provides a structured method for generating contracts, agreements, NDAs, leases, and related documents from user inputs. Editable previews and multi-format exports make the generated output easier to refine and reuse.

The project documentation concludes that LegalEase can help make legal information more accessible and understandable while supporting a modern, scalable application design. Continued development can extend document analysis, legal-data integration, personalization, and multilingual capabilities.

REFERENCES & PROJECT BASIS

This documentation is based primarily on the supplied LegalEase project document. The source describes the project purpose, scenarios, prerequisites, architecture, milestones, AI model selection, FastAPI routes, Streamlit interface, document formatting utilities, deployment workflow, testing steps, user operation, output formats, and conclusion.

The supplied material identifies FastAPI Documentation, Google Generative AI Documentation, Streamlit Documentation, and Python Official Documentation as prerequisite/reference resources.

TEAM DETAILS

TEAM ID: SWTID-2026-8781

TEAM SIZE: 5

TEAM LEADER: KOUSHIK S

TEAM MEMBERS: DHANUSH S, SARGUNANITHI S, HARRY RITHIK P, PRITAM DEBNATH